

Mini Torrent Share — Concept & Syntax Reference

A self-study companion to the project code. Each section covers **one concept used in the code**, the **core syntax**, a **minimal working example**, and **official docs** to go deeper.

1. WebRTC — the foundation

Concept

WebRTC lets two browsers open a **direct** connection to exchange data, audio, or video — without the data passing through a central server. Three pieces make this possible:

Term	What it does
ICE	Interactive Connectivity Establishment — the process of finding a usable network path between two peers (handles NAT/firewalls).
STUN server	A public server that tells your browser "here's your public IP/port as seen from the internet." Free public ones exist (e.g. Google's).
TURN server	A relay used only when a direct connection truly can't be established (symmetric NATs, strict firewalls). Traffic passes through it, so it's a fallback, not the default path.
RTCPeerConnection	The raw browser API representing the connection itself.
RTCDataChannel	A channel on that connection for sending arbitrary data (not just audio/video) — this is what carries our file chunks.

Why we didn't write raw WebRTC

Raw `RTCPeerConnection` requires manually exchanging "offer," "answer," and "ICE candidate" messages between peers via *some* side channel (a signaling server) before the direct connection opens. That handshake is correct but verbose. **PeerJS** (next section) wraps all of it.

Reference docs

- MDN WebRTC overview: https://developer.mozilla.org/en-US/docs/Web/API/WebRTC_API
 - MDN RTCDataChannel: <https://developer.mozilla.org/en-US/docs/Web/API/RTCDataChannel>
-

2. PeerJS — the library doing the handshake for us

Concept

PeerJS gives every browser tab a **Peer ID** and lets you call `.connect(otherId)` to open a WebRTC connection to another tab, using a small signaling server (our `server/index.js`) just to introduce them.

Core syntax

```
js
```

```
// Create a peer and register with the signaling server
const peer = new Peer(undefined, {
  host: 'localhost',    // your signaling server
  port: 9000,
  path: '/mini-torrent',
  secure: false,
});

// Fires once the server has assigned you an ID
peer.on('open', (id) => {
  console.log('My Peer ID is:', id);
});

// Fires if something goes wrong (server unreachable, ID taken, etc.)
peer.on('error', (err) => {
  console.error(err.type, err.message);
});

// Fires when ANOTHER peer initiates a connection to you
peer.on('connection', (conn) => {
  console.log('Incoming connection from', conn.peer);
});
```

js

```
// Actively connect to someone else's Peer ID
const conn = peer.connect('their-peer-id', { reliable: true });

conn.on('open', () => {
  conn.send({ hello: 'world' }); // send any JSON-serializable data
});

conn.on('data', (data) => {
  console.log('Received:', data);
});

conn.on('close', () => {
  console.log('Connection closed');
});
```

In our project: `app.js` calls `peer.connect(targetId)` in step 2 (the "Connect to someone" button), and `bindConnection()` attaches the same `open` / `data` / `close` handlers to whichever side initiated it.

Reference docs

- PeerJS docs: <https://peerjs.com/docs/>
- PeerServer (the Node half): <https://github.com/peers/peerjs-server>

3. Event-driven programming — `.on()` / `.emit()`

Concept

Instead of asking "did something happen yet?" in a loop, you **register a callback** for an event, and the library calls it when that event occurs. This is the same pattern as Node's `EventEmitter`, which you've already studied — PeerJS's `peer.on(...)` and `conn.on(...)` work identically.

Core syntax (Node's EventEmitter, for comparison)

```
js
```

```
const EventEmitter = require('events');
const emitter = new EventEmitter();

emitter.on('greet', (name) => {
  console.log(`Hello, ${name}!`);
});

emitter.emit('greet', 'Nusrat'); // triggers the listener above
```

PeerJS's `peer` and `conn` objects are effectively EventEmitters under the hood (adapted for the browser) — that's why `.on('open', ...)`, `.on('data', ...)`, `.on('close', ...)` all feel familiar.

Reference docs

- Node EventEmitter: <https://nodejs.org/api/events.html>

4. Chunking a file — the "torrent" part

Concept

Browsers can't just `send()` an arbitrarily large file in one WebRTC message reliably — large messages can fail or block the connection. So we slice the file into small pieces (**64KB each** in our code), send them one at a time tagged with an index, and reassemble them on the other end. This is the same underlying idea as **Streams**: process data in manageable pieces instead of loading everything into memory at once.

Core syntax

```
js

// Splitting a File/Blob into a chunk and reading its bytes
const file = fileInput.files[0];      // a File object
const CHUNK_SIZE = 64 * 1024;

const slice = file.slice(0, CHUNK_SIZE);      // Blob — just a "view"
const buffer = await slice.arrayBuffer();      // now actual bytes,
```

```
js
```

```
// Reassembling chunks back into a downloadable file
const chunks = [buffer1, buffer2, buffer3]; // ArrayBuffers, in
const blob = new Blob(chunks, { type: 'image/png' });
const url = URL.createObjectURL(blob);

// <a href={url} download="photo.png">Download</a>
```

In our project: `sendFile()` loops `while (offset < file.size)`, slicing and sending each piece; `handleIncomingData()` collects chunks into `incomingFile.chunks[msg.index]` and calls `finishReceivingFile()` once the last one arrives.

Reference docs

- MDN `Blob.slice()`: <https://developer.mozilla.org/en-US/docs/Web/API/Blob/slice>
- MDN `Blob.arrayBuffer()`: <https://developer.mozilla.org/en-US/docs/Web/API/Blob/arrayBuffer>
- MDN `Blob` constructor (for reassembly): <https://developer.mozilla.org/en-US/docs/Web/API/Blob/Blob>
- MDN `URL.createObjectURL`: https://developer.mozilla.org/en-US/docs/Web/API/URL/createObjectURL_static

5. `async`/`await` — waiting for slow things without blocking

Concept

Reading file bytes, or waiting for a network response, takes time. `async` functions let you write that waiting code top-to-bottom instead of nesting callbacks.

Core syntax

```
js

async function sendFile(file) {
  const buffer = await file.slice(0, 1024).arrayBuffer(); // pauses h
  console.log('Got', buffer.byteLength, 'bytes'); // ...then
}
```

```
js
```

```
// Yielding control back to the browser inside a loop
// (keeps the UI responsive during a long transfer)
await new Promise((resolve) => setTimeout(resolve, 0));
```

You've already covered `async`, `await`, `try/catch`, and Promises in your JS fundamentals — this project is the same pattern applied to file I/O instead of API calls.

Reference docs

- MDN async function: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Statements/async_function
 - MDN Promises guide: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Using_promises
-

6. Designing a tiny message protocol

Concept

Since `conn.send()` can carry any JSON-like object, we invent our own small "protocol" by tagging every message with a `kind` field, so the receiver knows how to handle it.

Core syntax (from `app.js`)

```
js
```

```
// Sender: describe the file before sending its bytes
conn.send({ kind: 'file-meta', name: 'cat.png', size: 204800, type: 'image/png' })

// Sender: send each piece with position info
conn.send({ kind: 'file-chunk', index: 0, total: 4, data: someArrayBuffer })
```

```
js
```

```
// Receiver: branch based on 'kind'
conn.on('data', (msg) => {
  if (msg.kind === 'file-meta') { /* prepare to receive */ }
  if (msg.kind === 'file-chunk') { /* store this piece */ }
});
```

This is the same idea as HTTP status codes or WebSocket message "types" — a lightweight, self-describing convention rather than a heavy framework.

7. Node.js server basics (recap, applied to `server/index.js`)

Core syntax

```
js

// CommonJS import (matches your recent CommonJS vs ESM study)
const { PeerServer } = require('peer');

// Start a server on a port, with options
const peerServer = PeerServer({
  port: 9000,
  path: '/mini-torrent',
});

// React to server-level events (same EventEmitter pattern as section
peerServer.on('connection', (client) => {
  console.log('Peer connected:', client.getId());
});
```

Reference docs

- Node.js `require`/CommonJS: <https://nodejs.org/api/modules.html>
 - PeerServer options: <https://github.com/peers/peerjs-server#options>
-

Suggested self-study order

1. Re-read `client/app.js` top to bottom with this doc open — every function maps to a section above.
2. Open two browser tabs locally, add a `console.log` inside each `peer.on(...)` / `conn.on(...)` handler, and watch the sequence of events fire in DevTools as you connect and send a file.
3. Try shrinking `CHUNK_SIZE` to something tiny (e.g. `4 * 1024`) and re-run a transfer — watch the progress bar update more often, which makes the chunking behavior very visible.
4. Once comfortable, attempt the "raw WebRTC" version mentioned in the project README (no PeerJS) as a stretch exercise — you'll be manually doing the offer/answer/ICE exchange that PeerJS currently hides from you.